

CS452: Intro to DL Internals

Assignment 2: The Magic Revealed - Backpropagation

Instructor: Ram Vikas, PhD

Objective

In the last session, we built a `Scalar` class that creates a computation graph. Today, we will implement the famous **backpropagation** algorithm. This algorithm is the engine of modern deep learning. Its job is to go "backward" through the graph we created and calculate the gradient of each `Scalar` with respect to the final output. We will use the **chain rule** from calculus to do this.

Part A: Understanding the Chain Rule (Conceptual Warm-up)

Reconsider our expression: $L = a \cdot b + c$. Let's set $e = a \cdot b$, so $L = e + c$. The chain rule tells us how to find the influence of `a` on `L`:

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e} \cdot \frac{\partial e}{\partial a}$$

Our task is to code this process. Backpropagation is just an algorithm for applying the chain rule automatically over our graph.

Part B: Implementing Backpropagation (Approx. 100 minutes)

To implement backpropagation, we need to add two key pieces of functionality to our `Scalar` class.

Task 1: The Local ‘`_backward`’ Function For each operation (`+`, `*`, `ReLU`), we need to define how it propagates gradients to its parents. This is the local application of the chain rule.

- In your `Scalar`'s `__init__` method, add a new attribute called `_backward`. Initialize it to a function that does nothing, for example: `lambda: None`.
- Modify your `__add__` method. After creating the `out` object, define a nested function inside `__add__` called `_backward()`.
 - This function's job is to update the gradients of the parents. For addition, the local derivatives are 1, so the rule is: `self.grad += out.grad` and `other.grad += out.grad`.
 - Assign this newly defined local function to the `out` object: `out._backward = _backward`.
- Modify your `__mul__` method similarly. The chain rule for multiplication $e = a \cdot b$ is $\frac{\partial e}{\partial a} = b$ and $\frac{\partial e}{\partial b} = a$. Your local `_backward` function should implement this: `self.grad += other.data * out.grad` and `other.grad += self.data * out.grad`.
- Modify your `relu` method. The derivative of ReLU is 1 if the input is positive, and 0 otherwise. Your local `_backward` function should implement this: `self.grad += (self.data > 0) * out.grad`.

Task 2: The ‘backward()’ Orchestrator Now we need a main method that calls all the local `_backward` functions in the correct order (from the end of the graph to the start).

- a. Add a new method to the `Scalar` class called `backward(self)`.
- b. **Topological Sort:** The first step is to get all nodes in the graph in a linear order.
 - Inside `backward`, create an empty list `topo` and an empty set `visited`.
 - Define a nested recursive function, `build_topo(v)`. This function should first check if a node `v` has been visited. If not, it should mark it as visited, then recursively call itself for all of `v`’s children (`v._prev`), and *finally* append `v` to the `topo` list.
 - Start the process by calling `build_topo(self)`.
- c. **Gradient Propagation:** After the `topo` list is built, you can go backward.
 - The gradient of the final node with respect to itself is 1. So, set `self.grad = 1.0`.
 - Iterate through the `topo` list in **reverse order**.
 - For each `node` in the reversed list, call its local backward function: `node._backward()`.

Task 3: Putting It All Together and Testing Let’s see if our implementation works!

- a. Assemble your complete `Scalar` class with all the new functionality.
- b. Re-run the test case from Assignment 1: create `Scalars` `a`, `b`, and `c`, and compute `L = a * b + c`.
- c. Call your new method: `L.backward()`.
- d. Print the `.grad` attributes of `a`, `b`, and `c`.
- e. **Verify:** Do the resulting gradients match the values you got from PyTorch in the first assignment? If so, congratulations!

Conclusion

You have just implemented the core algorithm behind all of modern deep learning. You’ve seen that backpropagation is not magic; it’s just an organized way of applying the chain rule over a computation graph. In our final session, we will use our `Scalar` class to build and train our very first neural network.